

SELENIUM WEBDRIVER

Introducción a la automatización de pruebas de aplicación Web

Conceptos fundamentales y configuración

Elaborado por:

David Quesada Calderón.

Fecha:

03 de septiembre de 2026

Herramienta:

Selenium 4.50.0

1. Selenium — Conceptos y fundamentos

1. Introducción

Selenium es un conjunto de herramientas utilizado principalmente para la automatización de pruebas de aplicaciones web.

Permite automatizar acciones que normalmente realizaría un usuario en un navegador, como:

- Abrir una página web.
- Ingresar información en formularios.
- Hacer clic en botones.
- Seleccionar opciones.
- Navegar entre páginas.
- Validar información mostrada en pantalla.
- Realizar procesos de inicio de sesión.
- Ejecutar casos de prueba de manera repetitiva.

2. Selenium WebDriver

El componente principal de Selenium utilizado para automatización es Selenium WebDriver.

WebDriver permite que un programa controle un navegador de forma automatizada.

3. Elementos Web

Para poder interactuar con una página, Selenium necesita identificar los elementos HTML.

Elementos comunes:

- `<input>`
- `<button>`
- `<select>`
- `<a>`
- `<textarea>`

4. Locators

Los locators son mecanismos utilizados para localizar elementos dentro de una página web.

Los principales son:

- ID
- `driver.find_element(By.ID, "username")`
- Name

- `driver.find_element(By.NAME, "username")`
- Class Name
- `driver.find_element(By.CLASS_NAME, "login-button")`
- Tag Name
- `driver.find_element(By.TAG_NAME, "button")`
- CSS Selector
- `driver.find_element(By.CSS_SELECTOR, "#username")`
- XPath
- `driver.find_element(By.XPATH, "//input[@id='username']")`

5. Métodos

Selenium proporciona diferentes métodos para interactuar con los elementos de una página web, como:

- Click
- Send Keys
- Clear
- Select
- Obtener texto
- Obtener atributos

6. Assertions

Las Assertions permiten comprobar que el resultado obtenido coincide con el resultado esperado.

Por ejemplo, después de iniciar sesión, se espera que aparezca el Dashboard:

- Si el elemento esperado aparece -> TEST PASS
- Si el elemento esperado no aparece -> TEST FAIL

7. Waits

Los Waits permiten el manejo de tiempos de espera.

Entre los principales se encuentran:

- Implicit Wait
- Explicit Wait

8. Page Object Model

En proyectos de automatización es recomendable utilizar patrones de diseño que permitan mantener el código organizado. Uno de los más utilizados con Selenium es Page Object Model (POM).

La idea es separar:

Pruebas

- Lógica de negocio

de:

Páginas

- Elementos
- Acciones

9. Flujo general de una prueba Selenium

Un flujo típico puede representarse así:

Inicio -> Abrir navegador -> Ingresar URL -> Localizar elemento -> Realizar acción -> Esperar respuesta -> Validar resultado -> PASS / FAIL -> Cerrar navegador

2. Configuración inicial de Selenium

1. Requisitos

Para comenzar a trabajar con Selenium se requiere instalar y configurar los siguientes componentes:

- Python.
- Selenium WebDriver.
- Un navegador web compatible, por ejemplo, Google Chrome.
- Un editor de código o IDE, como Visual Studio Code.

2. Verificar Python

El primer paso consiste en comprobar que Python se encuentra instalado correctamente.

En Windows se puede ejecutar desde PowerShell:

- `python --version`

Si Python está instalado correctamente, se mostrará la versión instalada

3. Crear un entorno virtual

Es recomendable utilizar un entorno virtual de Python para mantener aisladas las dependencias del proyecto.

Ejecutar:

- `python -m venv venv`

Para activar el entorno virtual en Windows:

- `.\venv\Scripts\Activate.ps1`

4. Instalar Selenium

Con el entorno virtual activado, instalar Selenium mediante `pip`:

- `pip install selenium`

5. WebDriver y los navegadores

Selenium puede gestionar la configuración necesaria para los drivers de los navegadores mediante Selenium Manager, por lo que en muchos escenarios no es necesario descargar y configurar manualmente `chromedriver`.

6. Crear archivo de dependencias

Para facilitar la instalación del proyecto en otros equipos, se recomienda crear un archivo de dependencias:

- `requirements.txt`

Posteriormente, las dependencias del proyecto pueden instalarse mediante:

- `pip install -r requirements.txt`

7. Estructura de un proyecto

proyecto-selenium

```
|
|
|— venv/
|
|
|— tests/
|   |— test_login.py
|
|
|— pages/
|   |— login_page.py
```

|
└── requirements.txt

8. Ejecutar el script

Desde PowerShell, dentro de la carpeta del proyecto:

- `python test_navegador.py`